System Architecture & Tech Stack Specification

STACK: Go 1.22+ | PostgreSQL 16 + SQLc | SvelteKit | Asynq + Redis BLUEPRINT v1.0

1. Architectural Vision: The Ticket Translator

The Intern Management System (IMS) is engineered to solve the disconnect between day-to-day technical task execution and corporate HR performance appraisals. Rather than forcing mentors to duplicate status updates across Jira and HR survey portals, the IMS operates as an event-driven **Kanban State Machine**. Every card transition is recorded in an immutable ledger, passively calculating cycle times and revision loops while actively intercepting anomalies to build a verifiable HR competency profile.

2. The Compiled "Geek" Stack vs. Legacy Runtimes

To win bottom-up adoption from DevOps engineers and Engineering Managers, the architecture rejects heavy interpreted runtimes (Python/Django, PHP, or Node.js ORMs) in favor of a compiled, statically typed, low-memory stack that deploys via a single `docker-compose.yml` manifest.

Architecture Layer	Selected Technology	Core Responsibility	Advantage Over Competitor Stacks
Backend Engine	**Go (Golang 1.22+)**	REST/SSE API, state machine validation, RBAC, and JWT license checks.	Compiles to a single <20MB static binary. Eliminates Node `node_modules` bloat and Python GIL bottlenecks.
Database & Access	**PostgreSQL 16 + SQLc**	ACID relational storage for the append-only transition ledger and HR rubrics.	SQLc compiles raw `.sql` queries into type-safe Go code at build time; zero ORM runtime overhead or schema drift vs. MongoDB.
Frontend UI	**SvelteKit**	Reactive Kanban drag-and-drop board, intercept modals, and HR dashboards.	Compiles components to surgical Vanilla JS without React's Virtual DOM overhead, delivering instant drag responsiveness.
Async Job Queue	**Asynq + Redis 7**	Hourly SLA threshold sweeps, webhook retries, and 30/60/90-day pulse locks.	Native Go worker pool with guaranteed retry semantics; eliminates manual Linux OS crontab configuration required by PHP/Kanboard.

Operational Footprint Target: The entire backend container runs on an `alpine` or `scratch` base image consuming under 25MB of idle RAM, enabling mid-market teams to self-host the complete stack on a $5/month cloud instance.

3. Relational Data Model & Append-Only Transition Ledger

Standard project management tools mutate a single `status` column on a task record, destroying historical timing context. The IMS treats task state changes as financial-grade ledger entries. Whenever a card moves across custom department columns, a database transaction updates the task pointer and inserts an immutable row into `transition_ledger`, computing the exact millisecond delta spent in the prior state.

```
      -- db/migrations/001_core_ledger.sql (Compiled via SQLc into internal/db/models.go)
CREATE TABLE kanban_columns ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), department_id
UUID NOT NULL REFERENCES departments(id), name VARCHAR(80) NOT NULL, -- e.g., 'Drafting',
'Awaiting MD Approval', 'UAT' position_index INT NOT NULL, -- Defines forward vs. backward
movement max_hours_in_state INT -- Nullable SLA threshold for neutral nudges ); CREATE TABLE
tasks ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), cohort_id UUID NOT NULL REFERENCES
cohorts(id), intern_id UUID  REFERENCES users(id), -- Nullable post-GDPR anonymization
mentor_id UUID NOT NULL REFERENCES users(id), skill_tag VARCHAR(50) NOT NULL, -- Competency
key: '#Python', '#DCF_Modeling' current_column_id UUID NOT NULL REFERENCES
kanban_columns(id), revision_count INT NOT NULL DEFAULT 0 ); CREATE TABLE transition_ledger
( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), task_id UUID NOT NULL REFERENCES tasks(id),
actor_id UUID NOT NULL REFERENCES users(id), from_column_id UUID REFERENCES
kanban_columns(id), to_column_id UUID NOT NULL REFERENCES kanban_columns(id),
time_in_prev_state_ms BIGINT NOT NULL DEFAULT 0, is_backward_bounce BOOLEAN NOT NULL DEFAULT
FALSE, transitioned_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

4. Background Threshold Worker (Asynq + Goroutines)

To stay department-agnostic across software, banking, and education cohorts, the backend makes zero assumptions about whether a slow task is the intern's fault or the mentor's fault. Instead, the Asynq scheduler enqueues a recurring `task:sla_sweep` job every 15 minutes:

- **Query Execution:** Executes a compiled SQLc query joining active `tasks` with their latest `transition_ledger.transitioned_at` timestamp against `kanban_columns.max_hours_in_state`.
- **Idempotent Notification Dispatch:** If elapsed time exceeds the column threshold and no alert has been sent for that transition ID, a worker Goroutine dispatches a neutral nudge via Slack/Email webhook so the mentor can intervene or approve the stalled work.

5. The "Justify This Action" Intercept Engine

Raw transition timestamps reveal *where* workflow friction occurred, but HR requires structured root-cause attribution to build a defensible evaluation dossier. When a mentor or intern drags a card on the SvelteKit Kanban board, the Go API evaluates the target column index against three deterministic friction triggers before committing the transaction:

- **Trigger A (Backward Revision Loop):** Target column `position_index < current_column.position_index` and `task.revision_count ≥ 1`.
- **Trigger B (Extreme Stagnation):** Time spent in the outgoing column exceeds 300% of `max_hours_in_state`.
- **Trigger C (Blocked Oscillation):** The task moves into a column flagged as `is_blocked_state` more than twice within 7 days.



Figure 1: Synchronous HTTP 428 (Precondition Required) intercept handshake capturing structured HR root-cause context at the moment of friction.

6. Explicit Domain Rules for HR 30/60/90-Day Appraisals

When an Asynq lifecycle job triggers a 30, 60, or 90-day review, the Go domain service aggregates `transition_ledger` and `intercept_events` using explicit fault-isolation rules:

- **Competency Warning Rule:** If `COUNT(KNOWLEDGE_GAP) ≥ 3` for a given `skill_tag` within the review window, the skill is flagged on the HR rubric for targeted pairing.
- **Fairness & Blocker Exclusion:** Any duration linked to `SCOPE_CREEP` or `THIRD_PARTY_BLOCKER` is mathematically excluded from the intern's cycle-time average, protecting interns from being penalized for mentor re-scoping or external IT delays.

7. Open-Core Monorepo Topology & Go Build Tags

The codebase is organized as a single Git monorepo that compiles into either the pure MIT Community Edition or the full Enterprise Edition using native Go build constraints (`//go:build ee`). In the community build, enterprise interfaces resolve to no-op stubs with zero proprietary dependencies; in the commercial build, the `ee/` package registers its middleware routes into the standard library HTTP multiplexer.

`cmd/ & internal/ (MIT Core)`

Contains `cmd/ims/main.go`, the Go 1.22 HTTP server, the agnostic Kanban state machine service (`internal/workflow/`), the intercept evaluator, and the Asynq background job handlers. 100% free and open-source.

`db/ & ui/ (MIT Core)`

`db/migrations/` and `db/query/` house the raw PostgreSQL schema and SQLc query definitions. `ui/` contains the SvelteKit frontend application compiled and embedded directly into the Go binary via `embed.FS`.

`ee/auth/ & ee/rbac/ (Enterprise)`

Compiled only when built with `go build -tags ee`. Provides enterprise SAML 2.0 and OIDC single sign-on, SCIM 2.0 automated cohort/mentor directory sync, and granular Policy-Based Access Control (PBAC).

`ee/audit/ & ee/license/ (Enterprise)`

Streams immutable compliance events to enterprise SIEM platforms (Datadog, Splunk, Elastic) and runs the RS256 public-key JWT verifier that validates offline license signatures and active intern seat limits.

8. Production Deployment & Architecture Decision Summary

Because the SvelteKit static bundle and SQL migrations are embedded directly inside the compiled Go binary using Go's native `embed` package, self-hosted deployment requires zero external web servers or Node.js containers in production. The entire production topology consists of just three containers orchestrated via Docker Compose: the stateless `ims-server` Go binary, `postgres:16-alpine`, and `redis:7-alpine`.

Key Architecture Decision Records (ADRs) Locked for MVP:

- **ADR-01:** Go 1.22 + SQLc selected over NestJS/Prisma for compile-time SQL safety and single-binary distribution.
- **ADR-02:** Append-only relational ledger in PostgreSQL selected over NoSQL documents for immutable HR auditability.
- **ADR-03:** Synchronous HTTP 428 intercept flow chosen over post-hoc survey forms to capture real-time mentor context.